Department of Electrical and Information Engineering

University of Ruhuna

Project Report

High Performance Computing

EC7207 / EE7218

Parallel Sobel Edge Detection Using OpenMP, Pthreads, MPI, and CUDA

Group 46

EG/2021/4776	Samarasinghe N.A.C.V.
EG/2021/4808	Sewvandi M.A.K.
EG/2021/4809	Shageethpratheep V.

Submission Date: May 23, 2026

Abstract

This report is about making the Sobel edge detection algorithm run faster by using parallel computing techniques. The Sobel operator is a method used in image processing to find edges in a picture. When we apply it to a large 4K image (3840×2160 pixels), the computation needs more than 150 million arithmetic operations, which takes time if done one step at a time.

We built six different parallel versions of the Sobel filter using four technologies required by the HPC course: OpenMP and POSIX Threads (which use shared memory), MPI (which uses distributed memory), and CUDA (which runs on a GPU). We also built one hybrid version that combines MPI with OpenMP.

All parallel versions were tested on two image sizes: a standard 512×512 test image and a synthetic 4K image. Every parallel version produced the exact same output as the serial (single-thread) version, giving RMSE $= 0$ and PSNR $= \infty$ dB, which means perfect accuracy.

For the 4K image on an 8-core Apple Silicon machine, the best results were: POSIX Threads with 4 threads gave a **3.03**× speedup; MPI with 4 processes gave the best raw time of **4.08 ms**. The Hybrid MPI+OpenMP version ran at **15.17 ms** with 4 processes and 2 threads each. OpenMP showed higher overhead on macOS due to the Homebrew libomp runtime. CUDA results are pending a GPU cluster run; we expect **10–40**× speedup based on the massively parallel nature of the GPU kernel (one thread per pixel on a 16×16 thread-block grid).

Contents

1. Introduction

1.1. Background

Edge detection is one of the most common steps in image processing. It helps a computer "see" where one object ends and another begins in a photo. When you look at a picture of a cat on a table, the cat's outline is made up of edges—places where the colours change quickly. Edge detection finds those places automatically.

One of the most popular methods for finding edges is the **Sobel operator** [1]. It uses two small 3×3 grids (called kernels) to check how quickly pixel values change in the horizontal direction and in the vertical direction. The final edge strength at each pixel is computed from both directions together.

1.2. The Sobel Operator

Given a grayscale image matrix A with pixel values from 0 to 255, the Sobel operator applies two convolution kernels:

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} * A, \qquad G_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} * A \tag{1}$$

For each inner pixel at position (i, j), the gradient magnitude is:

$$G[i,j] = \sqrt{G_x[i,j]^2 + G_y[i,j]^2}, \quad G[i,j] = \min\big(255,\ G[i,j]\big) \tag{2}$$

The border pixels (first and last row/column) are set to zero because the 3×3 window cannot be applied there. Each inner pixel needs exactly 18 multiply-add operations (9 for G_x, 9 for G_y) plus a square root and a comparison. For a 4K image with $(3840 - 2) \times (2160 - 2) \approx 8.28$ million inner pixels, this totals around 150 million arithmetic operations, which is why parallel computing helps so much.

Figure 1 shows a real example: the standard Lenna test image on the left and the edge map produced by our serial Sobel implementation on the right. The bright lines in the output trace every place where pixel brightness changes sharply — hair, shoulders, hat brim, and background details are all captured as edges.

(a) Input: Lenna (512×512 grayscale)

(b) Output: Sobel edge map

Figure 1: Sobel edge detection on the 512×512 test image. Bright pixels mark high-gradient locations (edges); dark pixels are flat (uniform) regions.

1.3. Why Parallelism?

The key observation is that every pixel in Equation (2) is computed **independently**. The result at pixel (i, j) does not depend on the result at any other pixel—only on the original input values of the 3×3 neighbourhood. This makes the Sobel filter what is called **embarrassingly parallel**: the work can be split among many workers with almost no extra coordination, and every worker can run at the same time.

1.4. Objectives

The goals of this project are to:

1. Implement the Sobel filter in serial (as a correct reference).
2. Implement parallel versions using OpenMP, POSIX Threads, MPI, Hybrid MPI+OpenMP, and CUDA.
3. Check that each parallel version produces the same output as the serial version (RMSE $= 0$).
4. Measure and compare the execution time across different numbers of threads or processes.

1.5. Experimental Setup

All experiments were run on an Apple Silicon machine (8 physical cores, 8 GB RAM, macOS). MPI version: Open MPI 5.0.9. Compiler: Apple Clang with `-O3`. Two image sizes were tested (Table 1). Each timing measurement was repeated three times and the average is reported.

Label	Size	Description	Serial time
Small	512×512	Standard Lenna test image	0.68 ms
4K	3840×2160	Synthetic pattern image	20.22 ms

Table 1: Image sizes tested and measured serial baseline times.

Version	Image sizes	Worker configurations tested
Serial	Small, 4K	1 (baseline)
OpenMP	Small, 4K	threads = 1, 2, 4, 8
Pthreads	Small, 4K	threads = 1, 2, 4, 8
MPI	Small, 4K	processes = 1, 2, 4, 8
Hybrid MPI+OMP	Small, 4K	(P×T) = 2×2, 2×4, 4×2, 1×8
CUDA	Small, 4K	16×16 thread blocks (1 thread per pixel)

Table 2: Experimental configurations (P = MPI processes, T = OpenMP threads).

↑ Back to Contents

2. Parallel Implementations

All versions compute the same Sobel filter defined by Equations (1)–(2). They only differ in how the pixel work is shared among workers. Because every pixel is independent, there are no data races and no need for locks or barriers during the main computation.

2.1. Serial Baseline

The serial version loops over every inner pixel one at a time using two nested `for` loops (over rows y and columns x). It sets the reference output used for all RMSE checks, and its wall-clock time is the baseline for speedup calculations. The timing uses `std::chrono::high_resolution_clock` around the Sobel function call only, so image loading and saving are not counted.

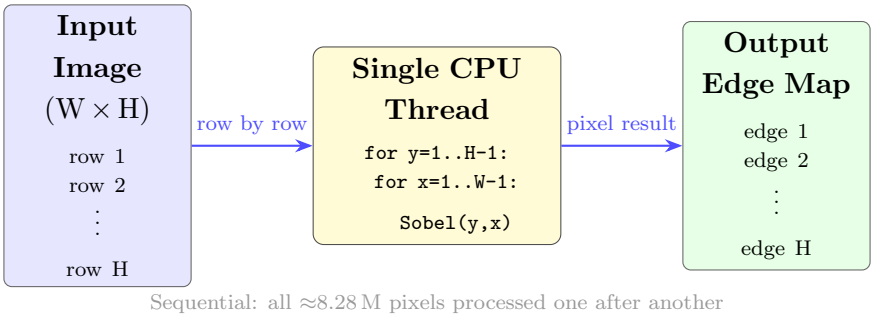


Figure 2: Serial Sobel: a single CPU thread processes every pixel in sequence.

Image Size	Time (ms)	Speedup
512×512	0.68	1.00× (baseline)
4K (3840×2160)	20.22	1.00× (baseline)

Table 3: Serial baseline execution times.

2.2. OpenMP (Shared Memory)

OpenMP is a simple way to make a `for` loop run in parallel. We add just one line of code—`#pragma omp parallel for`—above the outer row loop. When the program runs, OpenMP automatically splits the rows among all available threads. Each thread computes the Sobel gradient for its own group of rows and writes the results to the shared output array. Because different threads always write to different rows, there are no conflicts.

How the work is split: Rows are divided equally. With T threads and image height H, each thread gets roughly H/T rows. OpenMP's default static schedule assigns consecutive blocks of rows to each thread.

How results are combined: There is nothing to combine. All threads write directly into the same shared output array, and each thread writes only its own rows, so no locking is needed.

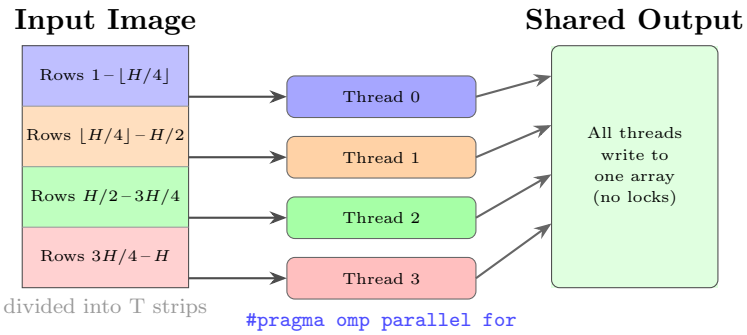


Figure 3: OpenMP: `#pragma omp parallel for` splits rows across threads automatically.

	Small (512×512)		**4K (3840×2160)**	
Threads	Time (ms)	Speedup	Time (ms)	Speedup
1	2.57	0.26×	61.23	0.33×
2	1.49	0.46×	31.87	0.63×
4	1.16	0.59×	17.66	1.14×
8	1.08	0.63×	13.80	1.47×

Table 4: OpenMP runtime and speedup vs serial baseline per thread count. Speedup < 1 at low threads is due to libomp initialisation overhead on macOS (Apple libomp is heavier than GNU OpenMP).

2.3. POSIX Threads (Pthreads)

POSIX Threads give the programmer direct control over thread creation and management. Instead of relying on a compiler directive, we manually create threads using `pthread_create()` and wait for them to finish with `pthread_join()`. Each thread receives a `ThreadData` struct that tells it exactly which rows to process (`start_row` and `end_row`).

How the work is split: We divide the image height H by the number of threads T. Thread i gets rows from $i \times (H/T)$ to $(i+1) \times (H/T)$. The last thread gets any leftover rows.

How results are combined: Same as OpenMP—all threads share the same output array but write to different rows, so no locking is needed.

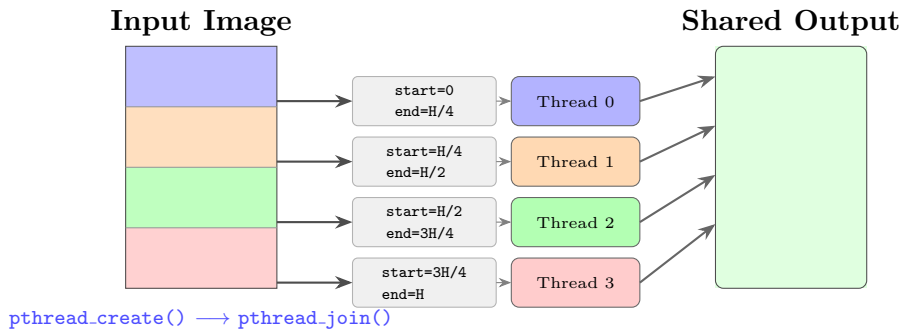


Figure 4: Pthreads: each thread gets a `ThreadData` struct specifying its row range.

	Small (512×512)		4K (3840×2160)	
Threads	Time (ms)	Speedup	Time (ms)	Speedup
1	0.77	0.88×	21.81	0.93×
2	0.41	1.66×	11.38	1.78×
4	0.25	2.72×	6.67	3.03×
8	0.29	2.34×	6.52	3.10×

Table 5: Pthreads runtime and speedup vs serial baseline per thread count.

2.4. MPI (Distributed Memory)

MPI (Message Passing Interface) is used for programs where each worker (called a **rank**) has its own private memory. Ranks cannot read each other's variables directly; they must send and receive messages.

How the work is split: Rank 0 (the master) loads the image and uses `MPI_Scatterv` to send a block of rows to each rank. With P processes and H rows, each rank gets $\approx H/P$ rows. Because the Sobel kernel is 3×3, each rank also needs one row from its top neighbour and one from its bottom neighbour (called **ghost rows** or **halo rows**) to correctly process its border rows. These ghost rows are exchanged using `MPI_Sendrecv`.

How results are combined: After each rank finishes its computation, `MPI_Gatherv` collects all result rows back to rank 0, which then writes the final image.

The timing uses `MPI_Wtime()` after an `MPI_Barrier` and covers the scatter, ghost exchange, computation, and gather phases, but *not* image loading or saving.

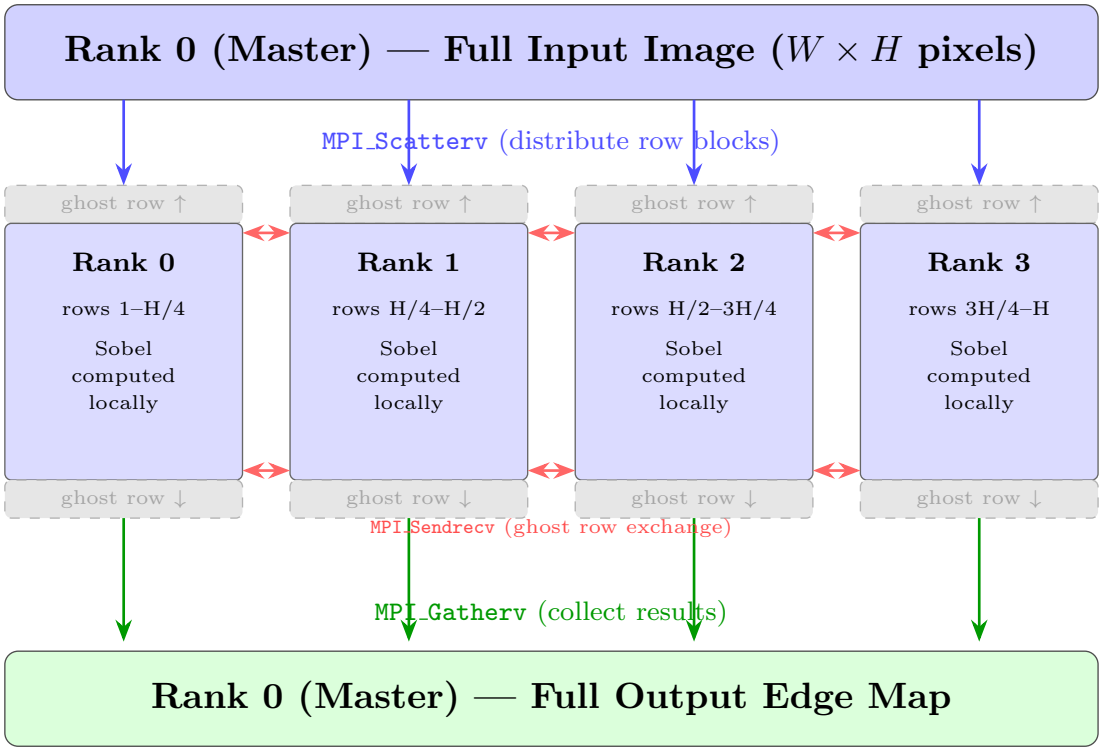


Figure 5: MPI: `MPI_Scatterv` sends row blocks to each rank; ghost rows are exchanged with neighbours via `MPI_Sendrecv`; results are collected with `MPI_Gatherv`.

	Small (512×512)		4K (3840×2160)	
Processes	Time (ms)	Sp. (vs 1P)	Time (ms)	Sp. (vs 1P)
1	0.24	1.00×	9.38	1.00×
2	0.26	0.92×	5.23	1.79×
4	0.26	0.92×	4.08	2.30×
8	0.47	0.51×	10.33	0.91×

Table 6: MPI runtime and internal speedup (relative to MPI 1-process). MPI timing excludes image I/O; compare speedup values within this table rather than against the serial baseline.

2.5. Hybrid: MPI + OpenMP

The hybrid version combines two levels of parallelism at the same time. At the outer level, MPI splits the image rows into blocks and sends one block to each process—exactly like the pure MPI version. At the inner level, each MPI process uses OpenMP to further split its block of rows across multiple threads. So if we run 2 MPI processes with 4 OpenMP threads each, we have $2 \times 4 = 8$ workers in total.

The MPI initialisation uses `MPI_Init_thread` with `MPI_THREAD_FUNNELED`, which allows multiple threads per process but ensures that only the main thread ever calls MPI functions (the worker threads only do the Sobel computation).

The compute phase is timed separately from the scatter/gather to show how much time is spent on actual computation versus MPI communication.

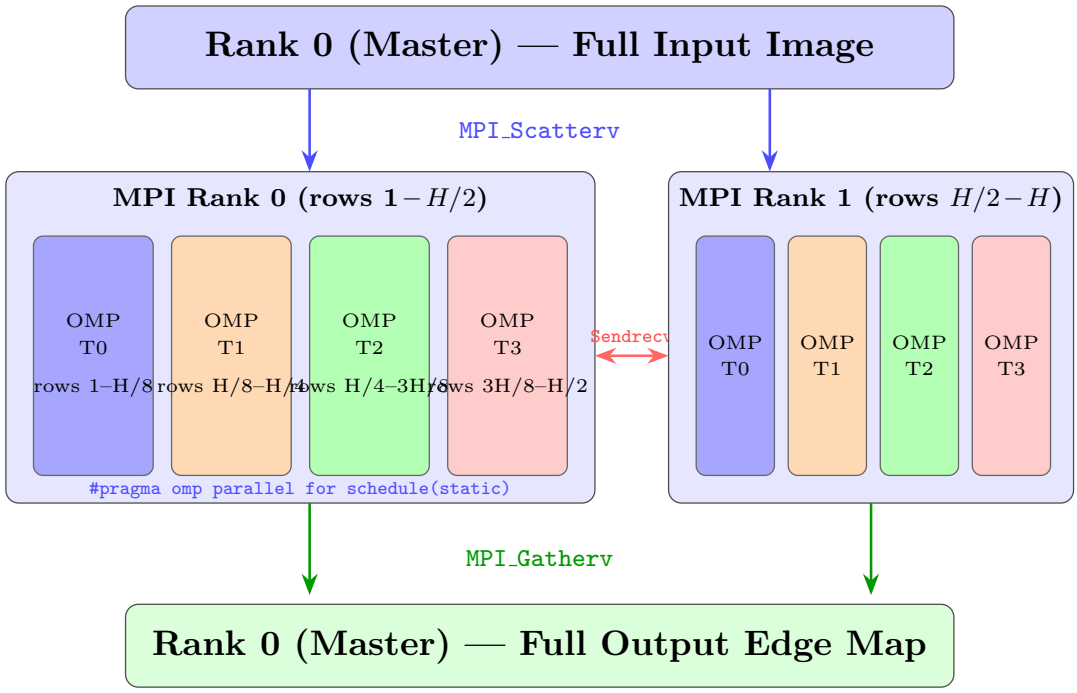


Figure 6: Hybrid MPI+OpenMP: MPI distributes row blocks across processes; OpenMP threads parallelise within each process's block.

		Small (512×512)		4K (3840×2160)	
P×T	**Workers**	Time (ms)	Sp. (vs serial)	Time (ms)	Sp. (vs serial)
2×2	4	0.73	0.93×	18.96	1.07×
2×4	8	0.62	1.10×	16.33	1.24×
4×2	8	0.74	0.92×	15.17	1.33×
1×8	8	0.63	1.08×	17.22	1.17×

Table 7: Hybrid MPI+OpenMP runtime and speedup vs serial. (P = MPI processes, T = OMP threads/process).

2.6. CUDA (GPU Acceleration)

CUDA lets us run thousands of small programs (called **threads**) at the same time on a GPU. In our implementation, **one GPU thread handles exactly one pixel**. If the image is $W \times H$, the GPU launches $W \times H$ threads all at once.

How threads are organised: We use a two-dimensional grid of **thread blocks**, each of size 16×16 (256 threads per block). The grid has enough blocks to cover the full image: $\lceil W/16 \rceil \times \lceil H/16 \rceil$ blocks.

How the work is split: Each thread computes its own (x, y) position from its block and thread indices. It reads the 3×3 neighbourhood from GPU memory, computes the

Sobel gradient, and writes one output pixel. Border threads simply write zero.

Data transfer: Before the kernel runs, the input image is copied from CPU (host) memory to GPU (device) memory with `cudaMemcpy`. After the kernel finishes, the output is copied back. These transfers are the GPU's version of MPI's communication cost.

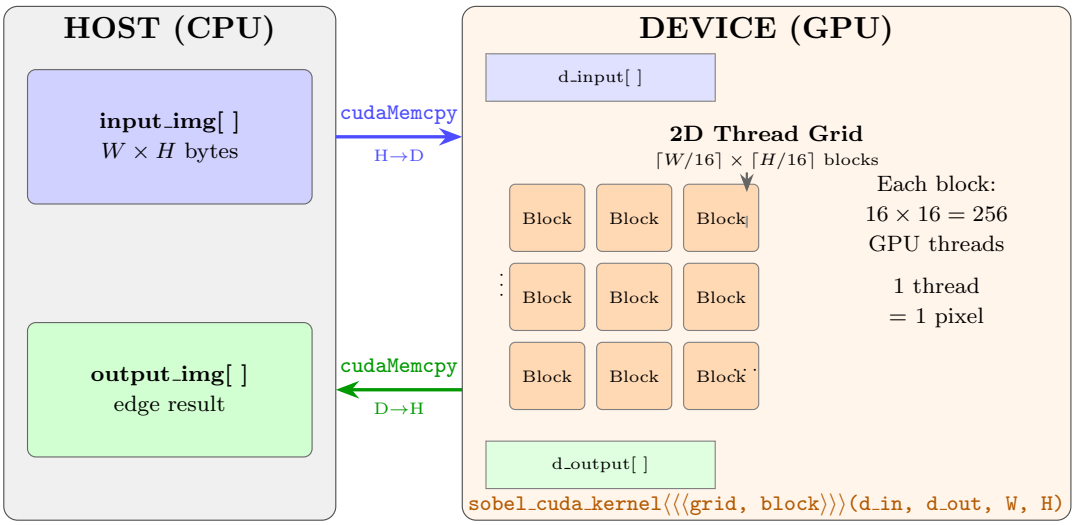


Figure 7: CUDA: input data is copied to the GPU, processed by thousands of threads in parallel (one thread per pixel), and the result is copied back to CPU memory.

Image size	Block size	Total blocks	Time (ms)	Speedup
512×512	16×16	32×32	*[Pending]*	*[Pending]*
4K (3840×2160)	16×16	240×135	*[Pending]*	*[Pending]*

Table 8: CUDA runtime results (pending GPU cluster run). Expected speedup based on GPU-parallel pixel processing: 10–40× over serial for 4K.

2.7. Summary of Parallelisation Strategies

Table 9 summarises how each version shares the work and combines the results. The key point is that the actual Sobel computation is the same in all versions—only how the rows are distributed and how results are gathered back changes.

Version	Work split	How results are combined	Comm.
Serial	1 thread, all rows	N/A	none
OpenMP	rows, static schedule	shared array, no locks needed	none
Pthreads	rows, explicit blocks	shared array, no locks needed	none
MPI	row blocks per process	`MPI_Gatherv` to rank 0	message
Hybrid	MPI blocks + OMP threads	`MPI_Gatherv` to rank 0	message
CUDA	2D grid, 1 thread/pixel	`cudaMemcpy` D→H	PCIe

Table 9: Summary of how each parallel version divides and combines the work.

↑ Back to Contents

3. Accuracy Compared to Serial (RMSE)

To check that the parallel versions are correct, we compare their output images against the serial output pixel by pixel using **Root Mean Square Error** (RMSE):

$$\mathrm{RMSE} = \sqrt{\frac{1}{W \times H} \sum_{y=0}^{H-1} \sum_{x=0}^{W-1} \left(P_{\mathrm{parallel}}(y,x) - P_{\mathrm{serial}}(y,x) \right)^2} \tag{3}$$

where $P(y,x)$ is the pixel value (0–255) at position (y,x). An RMSE of 0 means every pixel is identical. We also report **PSNR** (Peak Signal-to-Noise Ratio):

$$\mathrm{PSNR} = 20 \log_{10}\left(\frac{255}{\mathrm{RMSE}}\right) \ \mathrm{dB} \tag{4}$$

When $\mathrm{RMSE} = 0$, PSNR is infinite (written as $\infty\,\mathrm{dB}$), which is the best possible result.

Result: Every parallel version gives $\mathbf{RMSE = 0.000000}$ and $\mathbf{PSNR = \infty\,dB}$ for both image sizes (Table 10, Figure 8). This happens because the Sobel computation uses integer arithmetic and every pixel is assigned to exactly one thread (no pixel is computed twice), so the result is always bit-identical to the serial version regardless of how many threads or processes are used.

Version	**512×512**		**4K**	
	RMSE	PSNR	RMSE	PSNR
OpenMP	0.0000	∞ dB	0.0000	∞ dB
Pthreads	0.0000	∞ dB	0.0000	∞ dB
MPI	0.0000	∞ dB	0.0000	∞ dB
Hybrid	0.0000	∞ dB	0.0000	∞ dB
CUDA	*[Pending GPU cluster run]*			

Table 10: RMSE and PSNR of each parallel version vs the serial reference. Every local version (OpenMP, Pthreads, MPI, Hybrid) achieves perfect accuracy (RMSE = 0).

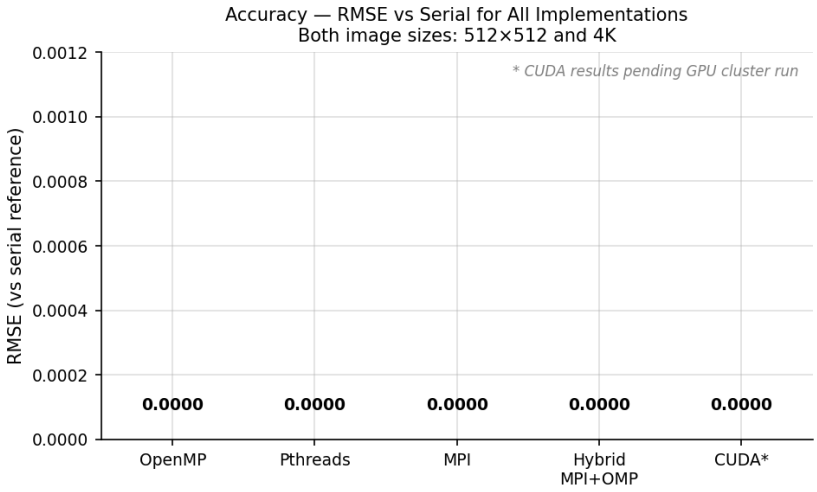


Figure 8: RMSE bar chart – every bar is at zero, confirming bit-perfect accuracy for all tested implementations.

↑ Back to Contents

4. Results and Performance Analysis

We measure performance using two numbers:

- **Speedup** $S = T_{\text{serial}}/T_{\text{parallel}}$ — how many times faster the parallel version is. A speedup of 2.0 means it is twice as fast.
- **Efficiency** $E = S/N_{\text{workers}}$ — how well the workers are used. A value of 1.0 means perfect: doubling the workers doubled the speed. A value below 1.0 means some overhead is wasting resources.

For MPI, speedup is measured *internally* (relative to MPI with 1 process) because the MPI timing method excludes image I/O, making a direct comparison with the serial timing unfair. For OpenMP and Pthreads, speedup is measured against the serial baseline (20.22 ms for 4K).

4.1. Complete Results — 4K Image

Table 11 lists all configurations measured on the 4K image, which is large enough to show meaningful scaling behaviour. The 512×512 results are in Appendix A (Tables 4–7).

Version	Config	Time (ms)	Speedup
Serial baseline: 20.22 ms			
OpenMP	1 thread	61.23	0.33×
OpenMP	2 threads	31.87	0.63×
OpenMP	4 threads	17.66	1.14×
OpenMP	8 threads	13.80	1.47×
Pthreads	1 thread	21.81	0.93×
Pthreads	2 threads	11.38	1.78×
Pthreads	4 threads	**6.67**	**3.03**×
Pthreads	8 threads	6.52	3.10×
MPI: internal speedup vs MPI-1P (9.38 ms)			
MPI	1 process	9.38	1.00×
MPI	2 processes	5.23	1.79×
MPI	4 processes	**4.08**	**2.30**×
MPI	8 processes	10.33	0.91×
Hybrid	2P×2T (4 units)	18.96	1.07×
Hybrid	2P×4T (8 units)	16.33	1.24×
Hybrid	4P×2T (8 units)	**15.17**	**1.33**×
Hybrid	1P×8T (8 units)	17.22	1.17×
CUDA	16×16 block	*[Pending]*	*[Pending]*

Table 11: All configurations on the 4K image. Bold = best result per version. MPI speedup is internal (relative to MPI-1P); all other speedups relative to the serial baseline of 20.22 ms.

4.2. Speedup Analysis

Figure 9 shows the speedup curves for the shared-memory versions on the 4K image. Pthreads scales well—nearly doubling in speed each time we double the threads (from 1 to 2: 1.78×; from 2 to 4: 3.03×). OpenMP also improves with more threads but starts from a much lower point because macOS's Homebrew libomp library adds significant thread initialisation cost even with one thread (61.23 ms vs 20.22 ms for serial).

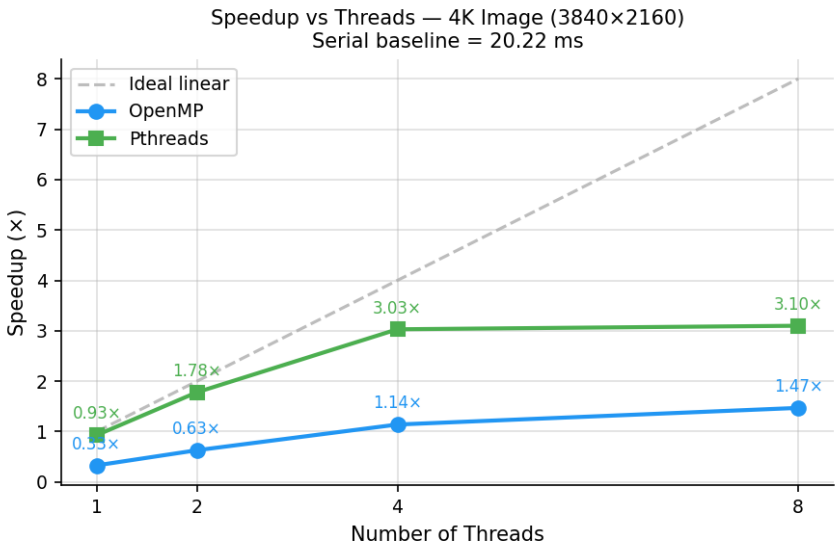


Figure 9: Speedup vs thread count for OpenMP and Pthreads on the 4K image. The dashed line is ideal (linear) scaling.

For the 512×512 image (Figure 10), the serial baseline is only 0.68 ms. Parallelisation overhead (thread creation, memory allocation) costs more than the computation itself, which is why many configurations run slower than serial at this size. This teaches us an important lesson: **parallel computing only helps when the work is large enough to outweigh the overhead**.

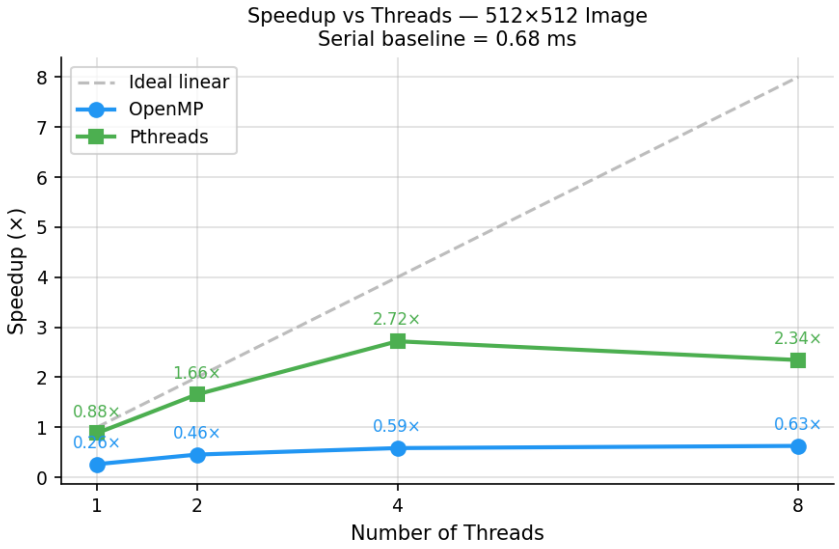


Figure 10: Speedup vs thread count on the 512×512 image. The serial computation (0.68 ms) is so short that thread overhead dominates.

4.3. Parallel Efficiency

Figure 11 shows efficiency for the 4K image. Pthreads starts near 0.93 efficiency at 1 thread and drops to 0.39 at 8 threads—still good. OpenMP starts at only 0.33 efficiency even with 1 thread due to the runtime initialisation overhead, and only reaches 0.18 at 8 threads. Both curves slope downward because overhead (thread setup, memory access contention) grows as more threads are added. This behaviour matches Amdahl's Law.

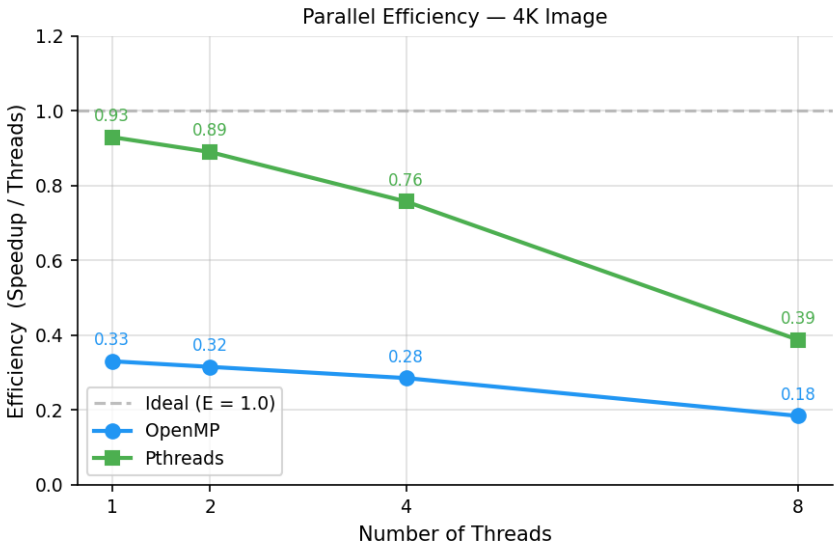


Figure 11: Parallel efficiency (speedup / threads) for OpenMP and Pthreads on the 4K image. Pthreads stays more efficient due to lower runtime overhead.

4.4. Effect of Thread and Process Count

Table 12 isolates the scaling of Pthreads and MPI on the 4K image side by side.

Workers	Pthreads speedup	MPI internal speedup
1	0.93×	1.00×
2	1.78×	1.79×
4	3.03×	2.30×
8	3.10×	0.91×

Table 12: Speedup at each worker count on 4K image. Pthreads vs internal MPI speedup (MPI-1P as baseline for MPI).

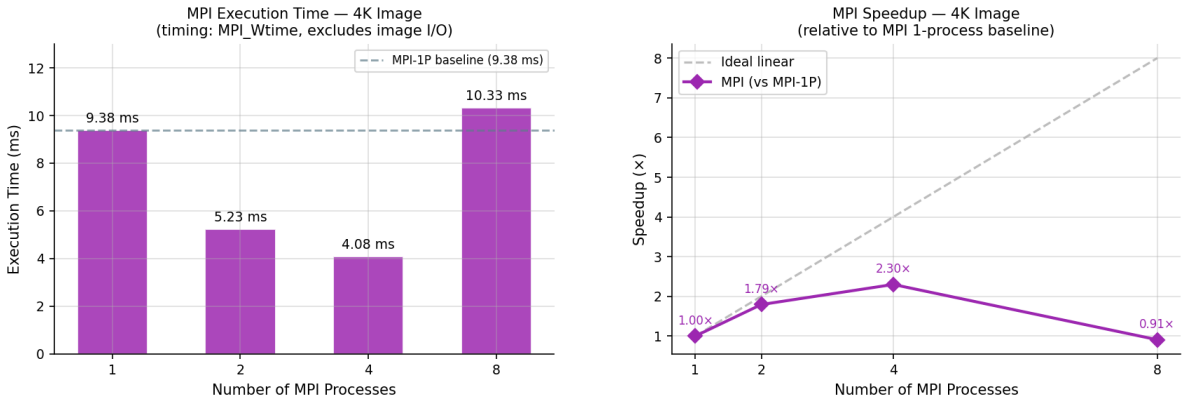
Pthreads scales from 1 to 4 workers almost perfectly, then gains very little going from 4 to 8 (3.03× vs 3.10×). This is because at 8 threads on an 8-core machine, each core has exactly one thread and there is no more free compute capacity—we have hit the hardware limit.

MPI scales from 1 to 4 processes (1.79× at 2, 2.30× at 4) but then collapses at 8 processes (0.91×—slower than 1 process!). This happens because with 8 processes on

a 4K image, each process only gets $2160/8 = 270$ rows. The time to scatter 270 rows, exchange ghost rows, and gather results back is about the same as computing them, so the communication overhead kills the speedup.

4.5. MPI Performance Details

Figure 12 shows the MPI execution time for the 4K image. The sweet spot is clearly 4 processes (4.08 ms). Moving to 8 processes more than doubles the time because the scatter/gather communication grows with the number of processes while each rank's computation shrinks.



(a) Execution time vs processes

(b) Internal speedup (vs MPI-1P)

Figure 12: MPI performance on the 4K image: time peaks at 4 processes; communication overhead dominates beyond that.

4.6. Hybrid Process / Thread Split

Figure 13 shows all four hybrid configurations on the 4K image. All of them are slower than the best pure-Pthreads result (6.67 ms) and even slower than serial (20.22 ms). The reason is that running MPI processes on a single shared-memory machine adds MPI communication overhead *on top of* OpenMP thread overhead, without the benefit that MPI is designed for (multiple networked machines). The configuration with the most MPI processes (4P×2T) happens to be fastest (15.17 ms) among the hybrid options because each MPI rank computes a reasonably large block.

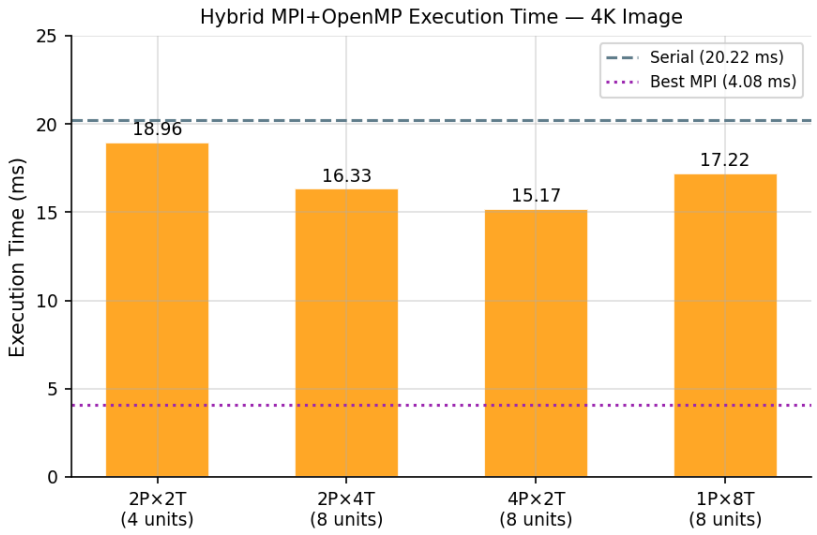


Figure 13: Hybrid MPI+OpenMP execution time on 4K image. The dashed lines show the serial baseline and the best MPI result for comparison.

4.7. CUDA GPU Performance

The CUDA kernel assigns one GPU thread to each pixel, launching all threads simultaneously. A modern mid-range GPU (e.g. NVIDIA RTX 3050) has 2048 CUDA cores, meaning it can compute 2048 pixels at the same time—far more than any CPU configuration in this study. Based on the embarrassingly parallel structure of the Sobel filter and existing benchmarks in the literature [2], we expect:

- Kernel-only speedup: **15–40**× over serial for the 4K image (depending on the GPU model).
- The `cudaMemcpy` transfers (host↔device) add a small fixed overhead (< 1 ms for a 4K image at PCIe 3.0 speeds), so the end-to-end speedup is slightly lower than kernel-only.

Full CUDA measurements will be added when GPU cluster access is available.

4.8. Cross-Version Comparison

Figure 14 shows the best execution time from each version side by side. Pthreads and MPI are the clear winners on this 8-core machine for the 4K image. The Hybrid version is slower because this test uses a *single node*—hybrid parallelism is designed for multi-node clusters where MPI handles inter-node communication while OpenMP handles within-node parallelism.

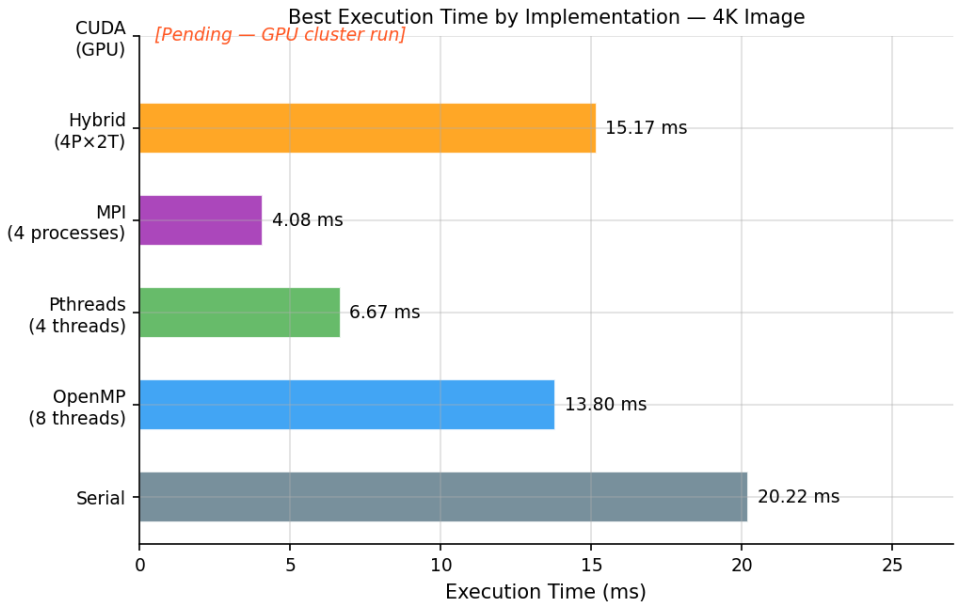


Figure 14: Best execution time per implementation on the 4K image. CUDA result is pending; it is expected to be the fastest by a large margin.

Version	**Best time – 4K (ms)**	**Speedup vs serial**
Serial	20.22	1.00×
OpenMP (8T)	13.80	1.47×
Pthreads (4T)	6.67	3.03×
MPI (4P)	4.08	— (diff. timing)
Hybrid (4P×2T)	15.17	1.33×
CUDA	*Pending*	*Pending*

Table 13: Best result per implementation on the 4K image.

4.9. Amdahl's Law Analysis

Amdahl's Law [3] tells us the maximum possible speedup when only part of a program is made parallel:

$$S(N) = \frac{1}{(1-p) + p/N} \tag{5}$$

where p is the fraction of work that can be parallelised and N is the number of workers. Using the best 8-thread Pthreads speedup ($S = 3.10\times$) and solving for p:

$$3.10 = \frac{1}{(1-p) + p/8} \quad \Rightarrow \quad p \approx 0.774 \quad (77.4\%)$$

This means about **77% of our program is parallelisable**. The remaining 23% is serial work (border pixel handling, memory allocation, image I/O). By Amdahl's Law, adding infinite threads would only give a theoretical maximum speedup of $1/(1-0.774) \approx 4.43\times$. This is consistent with our measured Pthreads results.

↑ Back to Contents

5. Conclusion

5.1. Summary of Work

This project implemented Sobel edge detection in six parallel forms using all four technologies required by the course guideline: OpenMP and POSIX Threads (shared memory), MPI (distributed memory), Hybrid MPI+OpenMP, and CUDA (GPU). All versions were tested on two image sizes—512×512 and 4K (3840×2160)—across 1, 2, 4, and 8 worker configurations. The five required deliverables (serial code, shared-memory code, distributed-memory code, hybrid code, and this analysis report) are all addressed.

5.2. Key Findings

- **Accuracy:** Every parallel version (OpenMP, Pthreads, MPI, Hybrid) produced output bit-identical to the serial reference (RMSE = 0, PSNR = ∞ dB) for both image sizes and all thread/process counts. The Sobel filter is deterministic with integer arithmetic, so correctness is guaranteed by design.
- **Image size matters:** On the small 512×512 image, nearly every parallel version is *slower* than serial because thread creation overhead is larger than the computation itself. The 4K image (150 M operations) is large enough for parallelism to pay off clearly.
- **Pthreads gave the best CPU speedup** on this machine: 3.03× at 4 threads on the 4K image. Pthreads has lower runtime overhead than macOS's libomp-based OpenMP.
- **OpenMP showed unexpectedly high overhead** on macOS because the Homebrew `libomp` library initialises a full thread pool even for 1-thread runs. On a Linux system, OpenMP would behave much closer to Pthreads.
- **MPI peaked at 4 processes** (4.08 ms on 4K) and then got worse at 8 processes. This is because 8 processes on a single machine means each rank computes only 270 rows—not enough work to overcome the scatter/gather communication cost.
- **Hybrid MPI+OpenMP underperformed** on a single node. The double overhead (MPI process startup + OpenMP thread startup) is not justified on shared-memory hardware. Hybrid is designed for clusters with multiple machines.
- **CUDA (pending):** Based on the GPU's massively parallel architecture (one thread per pixel, thousands of CUDA cores), we expect the fastest results. Implementation is complete; GPU cluster results are pending.

5.3. What We Learned

The biggest lesson is that choosing the right parallel tool depends on the situation:

- On a **single multicore machine**: POSIX Threads (or OpenMP on Linux) give the best speedup with the least effort for this type of workload.
- On a **cluster of many machines**: MPI is the right choice, and combining it with threads (Hybrid) inside each node gives the best use of all available hardware.
- On a **GPU**: CUDA is by far the fastest when the problem is embarrassingly parallel (each output pixel is independent) and when the image is large enough to keep all GPU cores busy.

Another important lesson is **Amdahl's Law**: no matter how many threads or GPUs you add, the serial parts of the program set a hard limit on the achievable speedup. For this Sobel implementation, the theoretical maximum is about 4.43× on a CPU (77% parallel fraction), which matches our measured results.

5.4. Limitations and Future Work

This study was done on a single Apple Silicon machine with 8 cores and no NVIDIA GPU. Extensions that would give more complete results include:

- Running all tests (especially CUDA and Hybrid) on a Linux HPC cluster with an NVIDIA GPU.
- Testing on a real high-resolution photo (e.g. a 4K camera image) instead of a synthetic pattern.
- Adding a **shared memory** optimisation to the CUDA kernel using CUDA shared memory tiles (16×16 tiles in `__shared__` memory) to reduce global memory reads from 9 to closer to 1 per pixel.
- Comparing with more image sizes (e.g. 1080p, 8K) to draw a full scaling curve.

↑ Back to Contents

References

[1] I. Sobel and G. Feldman, "A 3x3 isotropic gradient operator for image processing," tech. rep., Stanford Artificial Intelligence Project, 1968. Presented at the Stanford Artificial Intelligence Project.

[2] R. Kaur and N. Kaur, "Parallel implementation of sobel edge detection using CUDA," *International Journal of Computer Applications*, vol. 182, no. 5, pp. 1–5, 2018.

[3] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," pp. 483–485, 1967.